FINAL PROJECT GUIDELINE

Dr. Mansi Bhavsar

THE FINAL PROJECT

- You can work in teams of 1–3. Being in a team is encouraged.
- You can use any language/framework for your project
  - Though we expect nearly all of you to keep using python using google colab

PROJECT PROPOSAL – FROM EVERY TEAM 5%

1. Find a relevant (key) research paper for your topic
2. Write a summary of that research paper and what you took away from it as key ideas that you hope to use
3. Write what you plan to work on and how you can innovate in your final project work
   - Suggest a good milestone to have achieved as a halfway point

PROJECT MILESTONE – FROM EVERY TEAM 10%

- This is a progress report
- You should be more than halfway done!
- Describe the experiments you have run
- Describe the preliminary results you have obtained
- Describe how you plan to spend the rest of your time

You are expected to **have implemented some system** and to **have some initial experimental results** to show by this date (except for certain unusual kinds of projects)

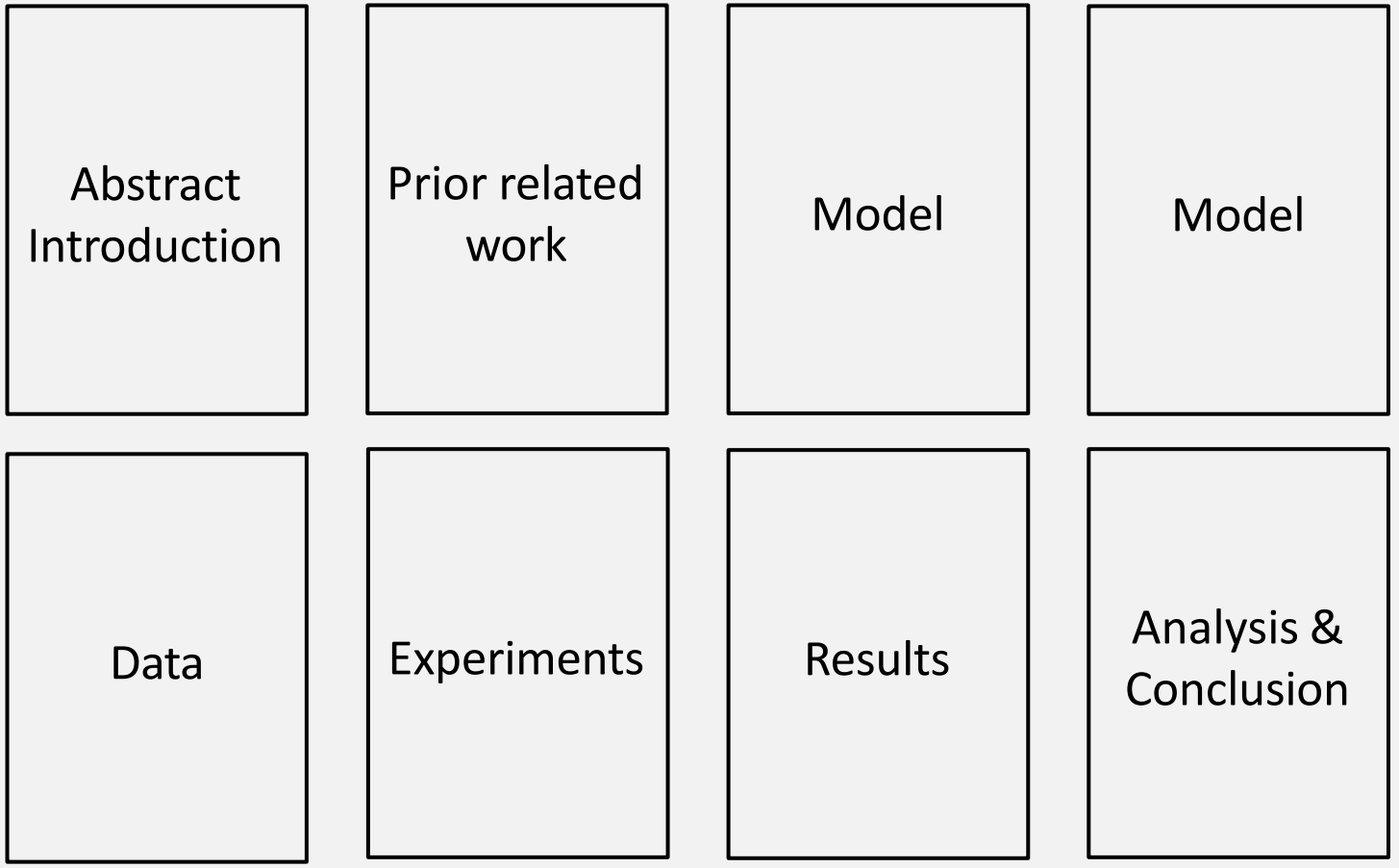
PROJECT PRESENTATION SLIDES FROM EVERY TEAM 15%

- Presentation quality is very important to your grade based on communication, presentation skills & content!!!

Abstract Introduction	Prior related work	Model	Model
Data	Experiments	Results	Analysis & Conclusion

PROJECT REPORT FROM EVERY TEAM 15%

- Writeup quality is very important to your grade!!! – It required to use IEEE format and it is given in the D2L



FINDING RESEARCH TOPICS

Two basic starting points, for all of science:

- [Nails] Start with a (domain) problem of interest and try to find good/better ways to address it than are currently known/used
- [Hammers] Start with a technical method/approach of interest, and work out good ways to extend it, improve it, understand it, or find new ways to apply it

PROJECT TYPES

This is not an exhaustive list, but most projects are one of

1. Find an application/task of interest and explore how to approach/solve it effectively, often with an existing model
   - Could be a task in the wild or some existing Kaggle/bake-off/shared task
   - Using API LLMs: be sure to go beyond just prompting a LM – we expect careful evals, comparison and analysis across multiple baselines.
2. Implement a neural architecture and demonstrate its performance on some data
3. Come up with a new or variant neural network model or approach and explore its empirical success
   - All the above are expected to have experiments with numbers and ablations ©
4. Analysis project. Analyze the behavior of a model: how it represents linguistic knowledge or what kinds of phenomena it can handle or errors that it makes
5. Rare theoretical or linguistic project: Show some interesting, non-trivial properties of a model type, data, or a data representation

Deep Poetry: Word-Level and Character-Level Language Models for Shakespearean Sonnet Generation

STANLEY XIE, RUCHIR RASTOGI AND MAX CHANG

Gated LSTM

Thy youth 's time and face his form shall cover?
Now all fresh beauty, my love there
Will ever Time to greet, forget each, like ever decease,
But in a best at worship his glory die.

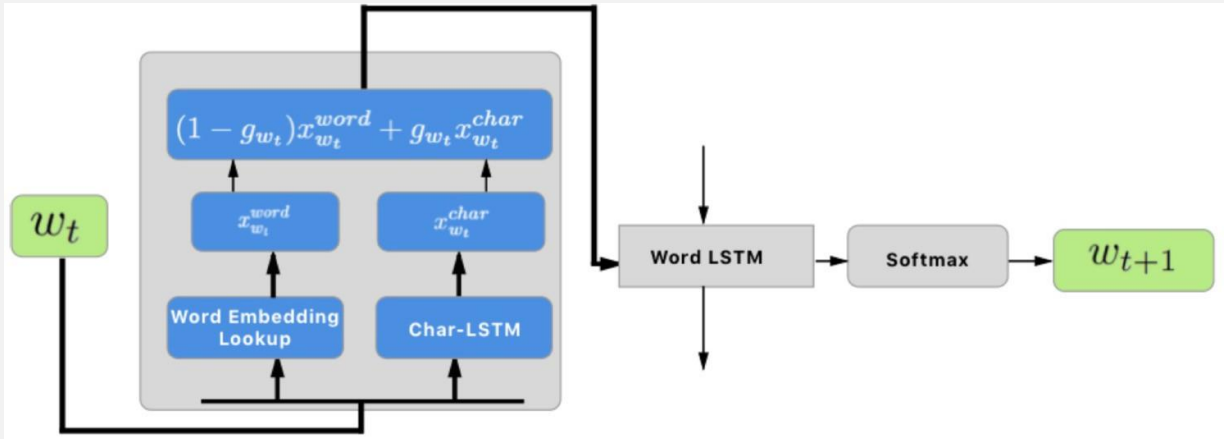
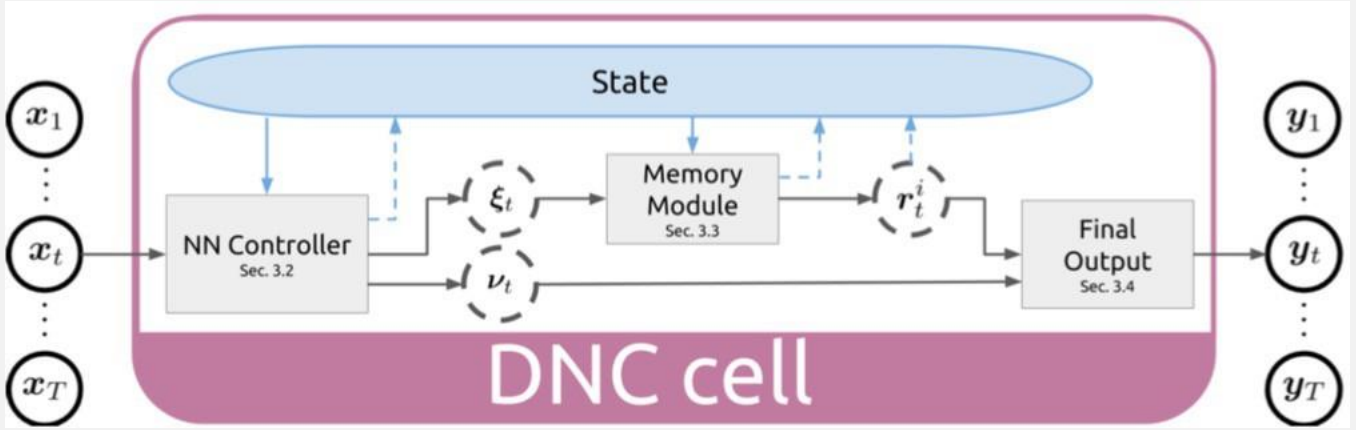


Figure 1: Architecture of the Gated LSTM

Implementation and Optimization of Differentiable Neural Computers

Carol Hsin
Graduate Student in Computational & Mathematical Engineering

We implemented and optimized Differentiable Neural Computers (DNCs) as described in the Oct. 2016 DNC paper [1] on the bAbI dataset [25] and on copy tasks that were described in the Neural Turning Machine paper [12]. This paper will give the reader a better understanding of this new and promising architecture through the documentation of the approach in our DNC implementation and our experience of the challenges of optimizing DNCs.



Improved Learning through Augmenting the Loss

Hakan Inan
inanh@stanford.edu

Khashayar Khosravi
khosravi@stanford.edu

We present two improvements to the well-known Recurrent Neural Network Language Models(RNNLM). First, we use the word embedding matrix to project the RNN output onto the output space and already achieve a large reduction in the number of free parameters while still improving performance. Second, instead of merely minimizing the standard cross entropy loss between the prediction distribution and the "one-hot" target distribution, we minimize an additional loss term which takes into account the inherent metric similarity between the target word and other words. We show with experiments on the Penn Treebank Dataset that our proposed model (1) achieves significantly lower average word perplexity than previous models with the same network size and (2) achieves the new state of the art by using much fewer parameters than used in the previous best work.

Published as a conference paper at ICLR 2017

Enhancing Self-Consistency and Performance of Pre-Trained Language Models through Natural Language Inference

Eric Mitchell, Joseph J. Noh, Siyan Li, William S. Armstrong, Ananth Agarwal, Patrick Liu, Chelsea Finn, Christopher D. Manning
Stanford University
eric.mitchell@cs.stanford.edu

Abstract

While large pre-trained language models are powerful, their predictions often lack logical consistency across test inputs. For example, a state-of-the-art Macaw question-answering (QA) model answers *Yes* to *Is a sparrow a bird?* and *Does a bird have feet?* but answers *No* to *Does a sparrow have feet?*. To address this failure mode, we propose a framework, Consistency Correction through Relation Detection, or **ConCoRD**, for boosting the consistency and accuracy of pre-trained NLP models using pre-trained natural language inference (NLI) models without fine-tuning or re-training. Given a batch of test inputs, ConCoRD samples several candidate outputs for each input and instantiates a factor graph that accounts for both the model's belief about the likelihood of each answer choice in isolation and the NLI model's beliefs about pair-wise answer choice compatibility. We show that a weighted MaxSAT solver can efficiently compute high-

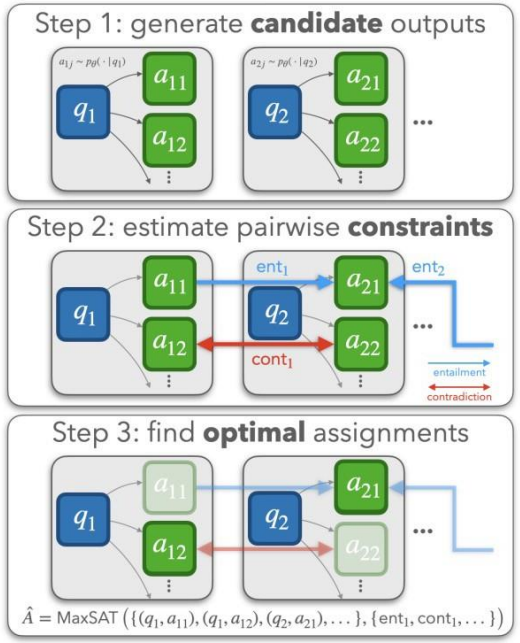


Figure 1: ConCoRD first generates candidate outputs from the base pre-trained model, then estimates soft pairwise constraints between output choices, and finally finds the most satisfactory choices of answers accounting for both the base model and NLI model's beliefs.

Merge of the work of two CS224N project teams, published at EMNLP 2022

Word2Bits - Quantized Word Vectors

Maximilian Lam
maxlam@stanford.edu

Abstract

Word vectors require significant amounts of memory and storage, posing issues to resource limited devices like mobile phones and GPUs. We show that high quality quantized word vectors using 1-2 bits per parameter can be learned by introducing a quantization function into Word2Vec. We furthermore show that training with the quantization function acts as a regularizer. We train word vectors on English Wikipedia (2017) and evaluate them on standard word similarity and analogy tasks and on question answering (SQuAD). Our quantized word vectors not only take 8-16x less space than full precision (32 bit) word vectors but also outperform them on word similarity tasks and question answering.

4. HOW TO FIND AN INTERESTING PLACE TO START?

- Look at ACL anthology for NLP papers:
  - https://aclanthology.org/
- Also look at the online proceedings of major ML conferences:
  - NeurIPS https://papers.nips.cc, ICML, ICLR https://openreview.net/group?id=ICLR.cc
- Look at online preprint servers, especially:
  - https://arxiv.org

- Even better: look for an interesting problem in the world!
  - Hal Varian: How to Build an Economic Model in Your Spare Time https://people.ischool.berkeley.edu/~hal/Papers/how.pdf

WANT TO BEAT THE STATE OF THE ART ON SOMETHING?

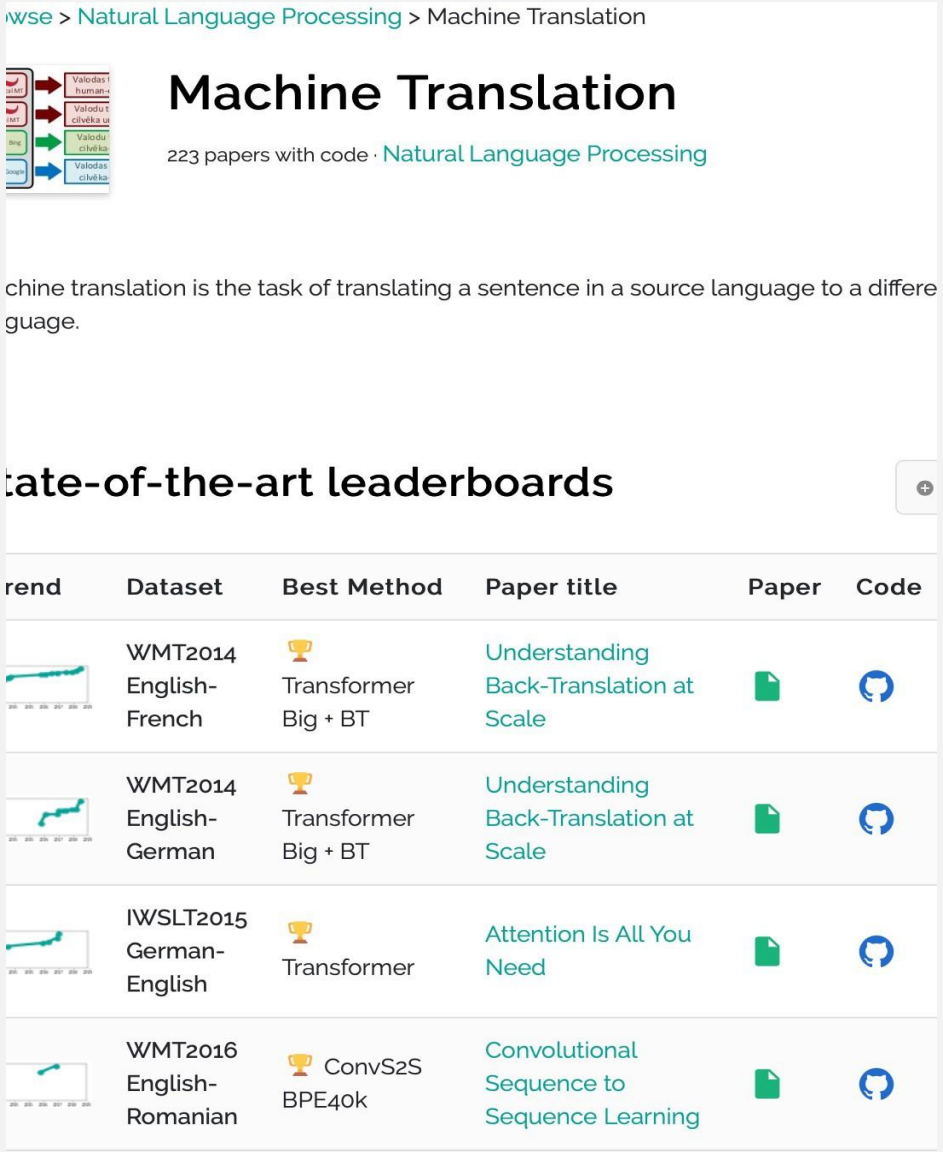
Great new sites that try to collate info on the state of the art

- Not always correct, though

https://paperswithcode.com/sota

https://nlpprogress.com/

Specific tasks/topics. Many, e.g.:
https://gluebenchmark.com/leaderboard/
https://www.conll.org/previous-tasks/



2024 NLP … RECOMMENDED FOR ALL YOUR PRACTICAL PROJECTS ©

```
pip install transformers   # By Huggingface 🤗

 # not quite runnable code but gives the general idea….

from transformers import BertForSequenceClassification, AutoTokenizer

model = BertForSequenceClassification.from_pretrained('bert-base-uncased’)

model.train()

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased’)

fine_tuner = Trainer( model=model,   args=training_args, train_dataset=train_dataset,
    eval_dataset=test_dataset )

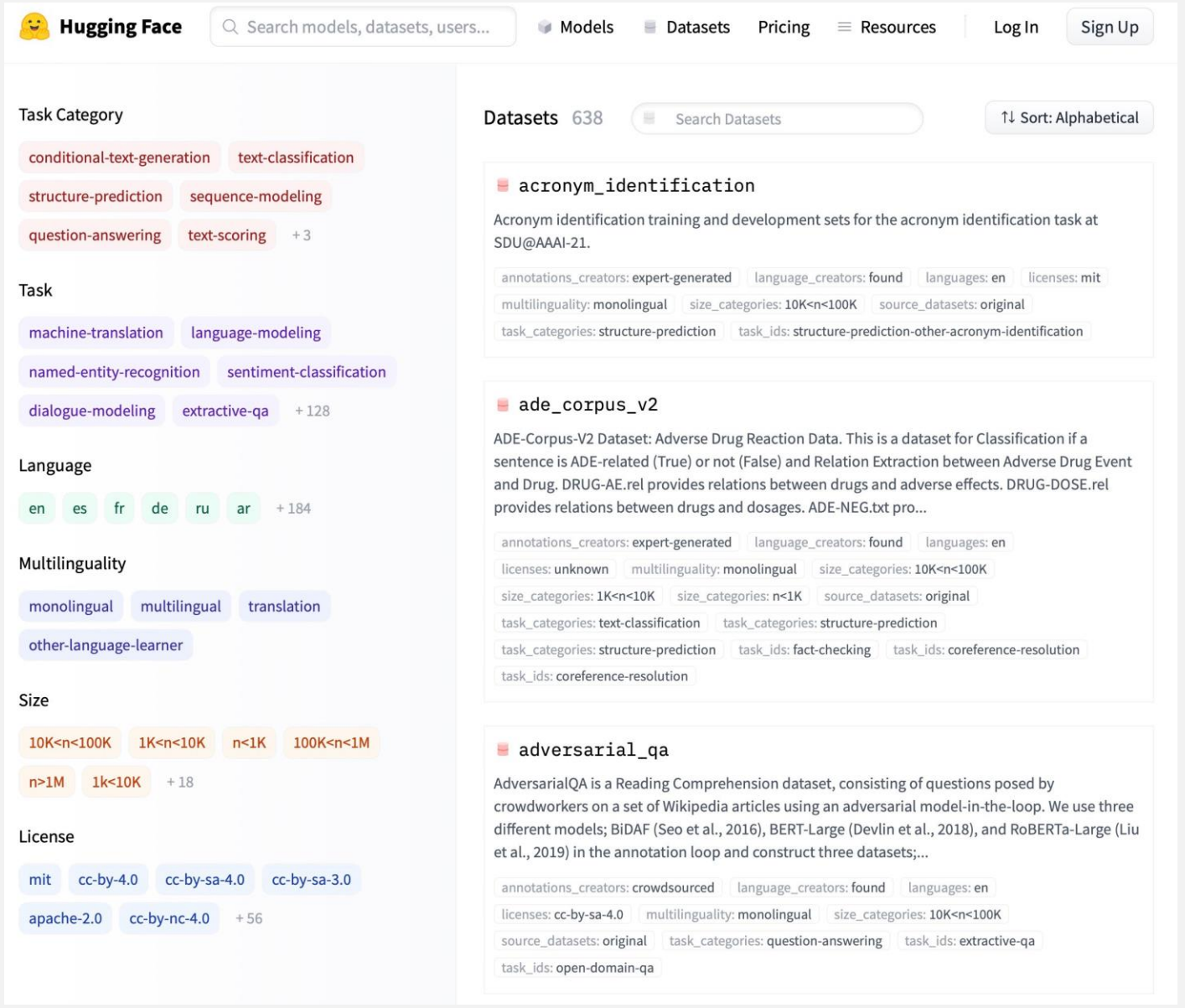
fine_tuner.train()

eval_dataset = load_and_cache_examples(args, eval_task, tokenizer, evaluate=True)

results = evaluate(model, tokenizer, eval_dataset, args)
```

HUGGINGFACE DATASETS

- https://huggingface.co/datasets



PAPERSWITHCODE DATASETS

- https://www.paperswithcode.com/datasets?mod=texts&page=1

835 dataset results for Texts ×



Penn Treebank

The English Penn Treebank corpus, and in particular the section of the corpus corresponding to the articles of Wall Street Journal (WSJ), is one of the most known and used corpus for t...

1,545 PAPERS • 10 BENCHMARKS



SQuAD (Stanford Question Answering Dataset)

The Stanford Question Answering Dataset (SQuAD) is a collection of question-answer pairs derived from Wikipedia articles. In SQuAD, the correct answers of questions can be any se-...

1,254 PAPERS • 7 BENCHMARKS



Visual Genome

Visual Genome contains Visual Question Answering data in a multi-choice setting. It consists of 101,174 images from MSCOCO with 1.7 million QA pairs, 17 questions per image on aver-...

903 PAPERS • 11 BENCHMARKS



GLUE (General Language Understanding Evaluation benchmark)

General Language Understanding Evaluation (GLUE) benchmark is a collection of nine natural language understanding tasks, including single-sentence tasks CoLA and SST-2, similarity...

847 PAPERS • 14 BENCHMARKS



SNLI (Stanford Natural Language Inference)

The SNLI dataset (Stanford Natural Language Inference) consists of 570k sentence-pairs manually labeled as entailment, contradiction, and neutral. Premises are image captions fro...

743 PAPERS • 1 BENCHMARK



CLEVR (Compositional Language and Elementary Visual Reasoning)

CLEVR (Compositional Language and Elementary Visual Reasoning) is a synthetic Visual Question Answering dataset. It contains images of 3D-rendered objects; each image comes...

528 PAPERS • 1 BENCHMARK



Visual Question Answering (VQA)

Visual Question Answering (VQA) is a dataset containing open-ended questions about images. These questions require an understanding of vision, language and commonsense...

435 PAPERS • 2 BENCHMARKS

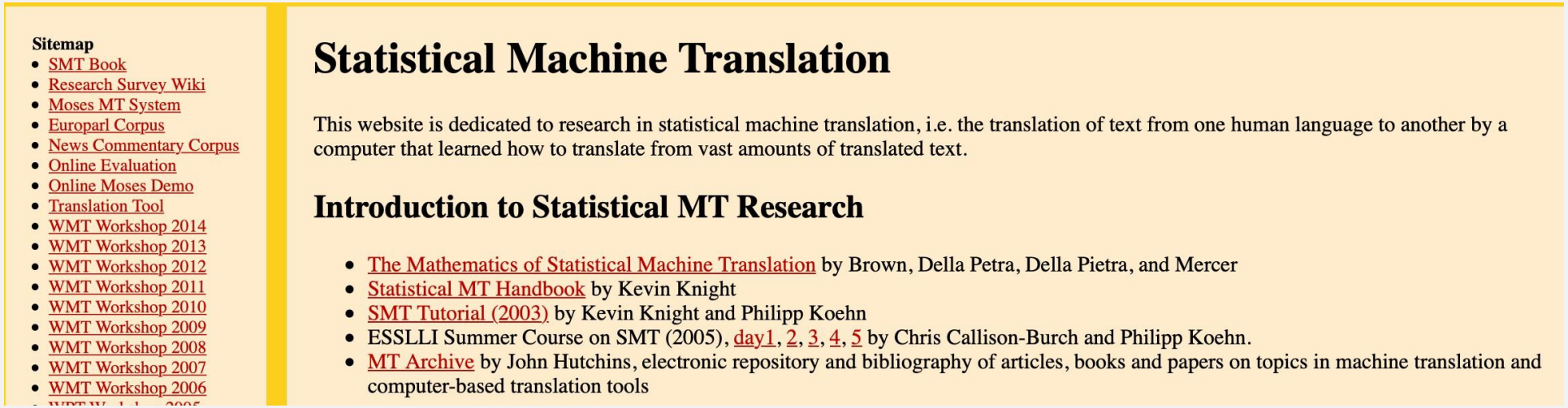
Billion Word Benchmark

The One Billion Word dataset is a dataset for language modeling. The training/held-out data was produced from the WMT 2011 News Crawl data using a combination of Bash shell and...

417 PAPERS • 1 BENCHMARK

MACHINE TRANSLATION

- http://statmt.org
- Look in particular at the various WMT shared tasks



Sitemap

- SMT Book
- Research Survey Wiki
- Moses MT System
- Europarl Corpus
- News Commentary Corpus
- Online Evaluation
- Online Moses Demo
- Translation Tool
- WMT Workshop 2014
- WMT Workshop 2013
- WMT Workshop 2012
- WMT Workshop 2011
- WMT Workshop 2010
- WMT Workshop 2009
- WMT Workshop 2008
- WMT Workshop 2007
- WMT Workshop 2006

Statistical Machine Translation

This website is dedicated to research in statistical machine translation, i.e. the translation of text from one human language to another by a computer that learned how to translate from vast amounts of translated text.

Introduction to Statistical MT Research

- The Mathematics of Statistical Machine Translation by Brown, Della Petra, Della Pietra, and Mercer
- Statistical MT Handbook by Kevin Knight
- SMT Tutorial (2003) by Kevin Knight and Philipp Koehn
- ESSLLI Summer Course on SMT (2005), day1, 2, 3, 4, 5 by Chris Callison-Burch and Philipp Koehn.
- MT Archive by John Hutchins, electronic repository and bibliography of articles, books and papers on topics in machine translation and computer-based translation tools

MANY, MANY MORE

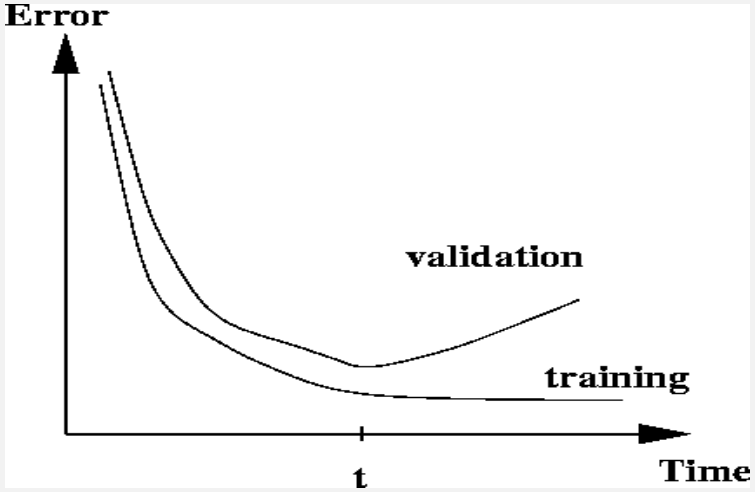
- There are now many other datasets available online for all sorts of purposes
  - Look at Kaggle
  - Look at research papers to see what data they use
  - Look at lists of datasets
    - https://machinelearningmastery.com/datasets-natural-language-processing/
    - https://github.com/niderhoff/nlp-datasets
  - Lots of particular things:
    - https://gluebenchmark.com/tasks
    - https://nlp.stanford.edu/sentiment/
    - https://research.fb.com/downloads/babi/ (Facebook bAbI-related)

5. CARE WITH DATASETS AND IN MODEL DEVELOPMENT

- Many publicly available datasets are released with a **train/dev/test** structure.
- **We're all on the honor system to do test-set runs only when development is complete.**
- Splits like this presuppose a fairly large dataset.
- If there is no dev set or you want a separate tune set, then you create one by splitting the training data
  - We have to weigh the usefulness of it being a certain size against the reduction in train-set size.
  - Cross-validation (q.v.) is a technique for maximizing data when you don't have much
- Having a fixed test set ensures that all systems are assessed against the same gold data. This is generally good, but it is problematic when the test set turns out to have unusual properties that distort progress on the task.

TRAINING MODELS AND POTS OF DATA

- When training, models **overfit** to what you are training on
  - The model correctly describes what happened to occur in particular data you trained on, but the patterns are not general enough patterns to be likely to apply to new data
- The way to monitor and avoid problematic overfitting is using **independent** validation and test sets …



TRAINING MODELS AND POTS OF DATA

- You build (estimate/train) a model on a **training set**.
- Often, you then set further hyperparameters on another, independent set of data, the **tuning set**
  - The tuning set is the training set for the hyperparameters!
- You measure progress as you go on a **dev set** (development test set or validation set)
  - If you do that a lot you overfit to the dev set so it can be good to have a second dev set, the **dev2** set
- **Only at the end**, you evaluate and present final numbers on a **test set**
  - Use the final test set **extremely** few times … ideally only once

GETTING YOUR NEURAL NETWORK TO TRAIN

- Start with a positive attitude!
  - **Neural networks want to learn!**
    - If the network isn't learning, you're doing something to prevent it from learning successfully

- Realize the grim reality:
  - **There are lots of things that can cause neural nets to not learn at all or to not learn very well**
    - Finding and fixing them ("debugging and tuning") can often take more time than implementing your model

- It's hard to work out what these things are
  - But experience, experimental care, and rules of thumb help!

EXPERIMENTAL STRATEGY

- Work incrementally!
- Start with a very simple model and get it to work!
  - It's hard to fix a complex but broken model
- Add bells and whistles one-by-one and get the model working with each of them (or abandon them)

- Initially run on a tiny amount of data
  - You will see bugs much more easily on a tiny dataset … and they train really quickly
  - Something like 4–8 examples is good
  - Often synthetic data is useful for this
  - Make sure you can get 100% on this data (testing on train)
    - Otherwise your model is definitely either not powerful enough or it is broken

EXPERIMENTAL STRATEGY

- Train and run your model on a large dataset
  - It should still score close to 100% on the training data after optimization
    - Otherwise, you probably want to consider a more powerful model!
    - Overfitting to training data is **not** something to fear when doing deep learning
      - These models are usually good at generalizing because of the way distributed representations share statistical strength regardless of overfitting to training data
- But, still, you now want good generalization performance:
  - Regularize your model until it doesn't overfit on dev data
    - Strategies like L2 regularization can be useful
    - But normally **generous dropout** is the secret to success

DETAILS MATTER!

- Look at your data, collect summary statistics
- Look at your model's outputs, do error analysis
- Tuning hyperparameters, learning rates, getting initialization right, etc. is **often** important to the successes of neural nets